

گزارش فاز اول پروژه هوش مصنوعی

مسیر ۱ Agent-Based AI :

عنوان پروژه: طراحی عامل هوشمند Pac-Man با استفاده از الگوریتم Minimax

تعریف مسئله

در این پروژه هدف طراحی و پیاده‌سازی یک عامل هوشمند (Intelligent Agent) برای بازی Pac-Man در یک محیط Grid-Based است.

در این محیط:

- Pac-Man باید تمام نقاط غذا (Food Points) را جمع‌آوری کند.
- دو Ghost به عنوان عامل‌های دشمن حضور دارند.
- اگر Pac-Man با یکی از Ghost ها برخورد کند، بازی پایان می‌یابد.
- اگر تمام نقاط جمع‌آوری شوند، Pac-Man برنده است.

تعریف دقیق مسئله

هدف طراحی یک عامل تصمیم‌گیرنده مبتنی بر الگوریتم Minimax است که:

- در هر مرحله بهترین حرکت را انتخاب کند.
- از Ghost ها فاصله ایمن حفظ کند.
- در عین حال مسیر بهینه برای رسیدن به نزدیک‌ترین غذا را انتخاب کند.
- بتواند در عمق جستجوی مشخص ($MAX_DEPTH = 9$) تصمیم‌گیری انجام دهد.

مدل‌سازی محیط

محیط بازی شامل:

- ابعاد:

- BOARD_WIDTH
- BOARD_HEIGHT
- موانع (OBSTACLES)
- موقعیت اولیه Pacman
- موقعیت اولیه دو Ghost
- مجموعه‌ای از نقاط غذا قابل جمع‌آوری

فضای حالت (State Space)

هر وضعیت بازی شامل موارد زیر است:

GameState:

-موقعیت Pacman

-موقعیت Ghost ها

-نقاط باقی‌مانده

-امتیاز فعلی

فضای حالت به صورت:

$$S = (\text{Pacman_position}, \text{Ghost_positions}, \text{Remaining_points}, \text{Score})$$

تعریف شده است.

نوع عامل (Agent Type)

عامل طراحی شده:

Utility-Based Agent

Adversarial Agent

Multi-Agent Environment

چون:

- Pacman به دنبال بیشینه‌سازی Utility است.

- Ghost ها نقش Min-Agent دارند.
- محیط رقابتی (Competitive) است.

الگوریتم انتخاب شده

برای تصمیم گیری از الگوریتم:

Minimax Algorithm

استفاده شده است.

ساختار تصمیم گیری:

- Agent 0 → Pacman (Max Player)
- Agent 1 → Ghost 1 (Min Player)
- Agent 2 → Ghost 2 (Min Player)

چرخه تصمیم گیری:

Pacman → Ghost1 → Ghost2 → Pacman → ...

عمق جستجو:

MAX_DEPTH = 9

تابع Utility طراحی شده

تابع Utility مهم ترین بخش این پروژه است.

در این پروژه Utility شامل:

1 وضعیت پایانی

- اگر Pacman توسط Ghost گرفته شود $-\infty$ →
- اگر همه غذاها جمع شوند $+\infty$ →

2 فاصله از Ghost ها

با استفاده از: Manhattan Distance

$$\text{distance} = |x_1 - x_2| + |y_1 - y_2|$$

قوانین امتیازدهی:

- فاصله $+100 \rightarrow > 3$
- فاصله $-1000 \rightarrow < 2$
- فاصله متوسط $\rightarrow -100$

3 فاصله تا نزدیک‌ترین غذا

با استفاده از BFS کوتاه‌ترین مسیر تا غذا محاسبه می‌شود.

هرچه Pacman در مسیر بهینه‌تری باشد:

$$I = \text{distance_to_food} * 150$$

فرمول نهایی: Utility

$$\text{Utility} = \text{score} + k + I$$

طراحی BFS برای مسیر غذا

برای پیدا کردن کوتاه‌ترین مسیر تا غذا از الگوریتم:

Breadth First Search (BFS)

استفاده شده است.

BFS:

- از موقعیت فعلی شروع می‌کند
- اولین غذایی که پیدا کند مسیر را برمی‌گرداند
- تضمین می‌کند کوتاه‌ترین مسیر انتخاب شود

ساختار کلی پروژه

src/
game_state.py
minimax.py
utils.py

main.py
README.md

تحلیل پیچیدگی

اگر:

• $b = \text{branching factor}$

• $d = \text{depth}$

پیچیدگی زمانی: Minimax

$$O(b^d)$$

در این پروژه:

• $b \approx 5$

• $d = 9$

که حدوداً:

$$5^9 \approx 1,953,125$$

حالت ممکن را بررسی می کند.

به همین دلیل عمق 9 انتخاب شده که تعادل بین کیفیت تصمیم و زمان اجرا باشد.

چالش های مسئله

1. فضای حالت بزرگ

2. وجود دو Min-Agent

3. طراحی Utility مناسب

4. جلوگیری از Loop شدن Pacman

5. تعادل بین فرار از Ghost و جمع‌آوری غذا

مدل پایه (Baseline)

مدل پایه در این پروژه:

Pacman با انتخاب حرکت تصادفی (Random Agent)

مقایسه عملکرد:

نتیجه	مدل
شکست سریع	Random
تصمیم هوشمند، بقا بیشتر	Minimax

پلن آزمایش

آزمایش‌های طراحی شده:

- تست عمق‌های مختلف (5, 7, 9)
- تست وزن‌دهی‌های مختلف Utility
- مقایسه با Random Agent
- بررسی تاثیر فاصله از Ghost در عملکرد

معیار ارزیابی:

- مدت زمان بقا
- امتیاز نهایی
- تعداد غذاهای جمع‌آوری شده